

Recursividade

O que é Recursividade?

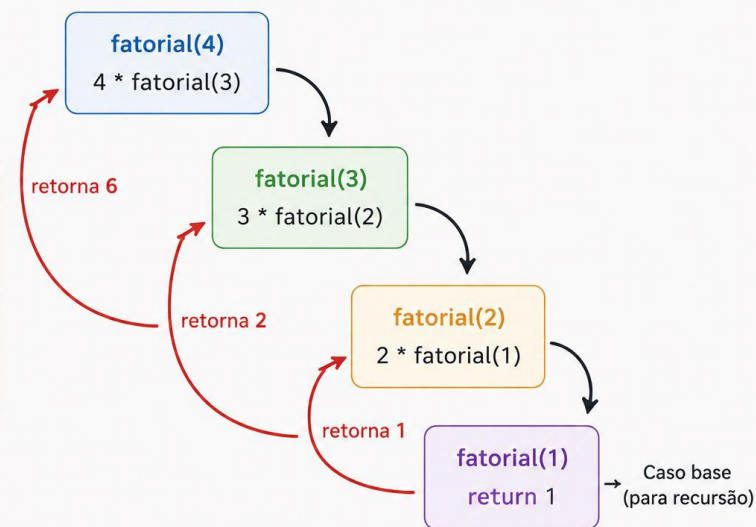
Na programação em C, a recursividade ocorre quando uma função resolve um problema chamando **a si mesma**.

Em vez de usar laços de repetição, dividimos o problema em subproblemas idênticos, mas de tamanho menor, até atingirmos uma condição trivial e conhecida.

Visualmente, é como estar entre dois espelhos: a imagem se repete infinitamente até esbarrar em um limite prático.

Chamadas recursivas

```
int fatorial(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fatorial(n - 1);  
}
```



Um Exemplo **Simple**s em C

A Função de Contagem

Veja como declaramos uma função que chama a si mesma. Este programa realiza uma contagem regressiva simples.

Para não rodar infinitamente, adicionamos uma condição de parada, que chamamos formalmente de **caso base**.

Aqui, a parada ocorre quando o número atinge zero.

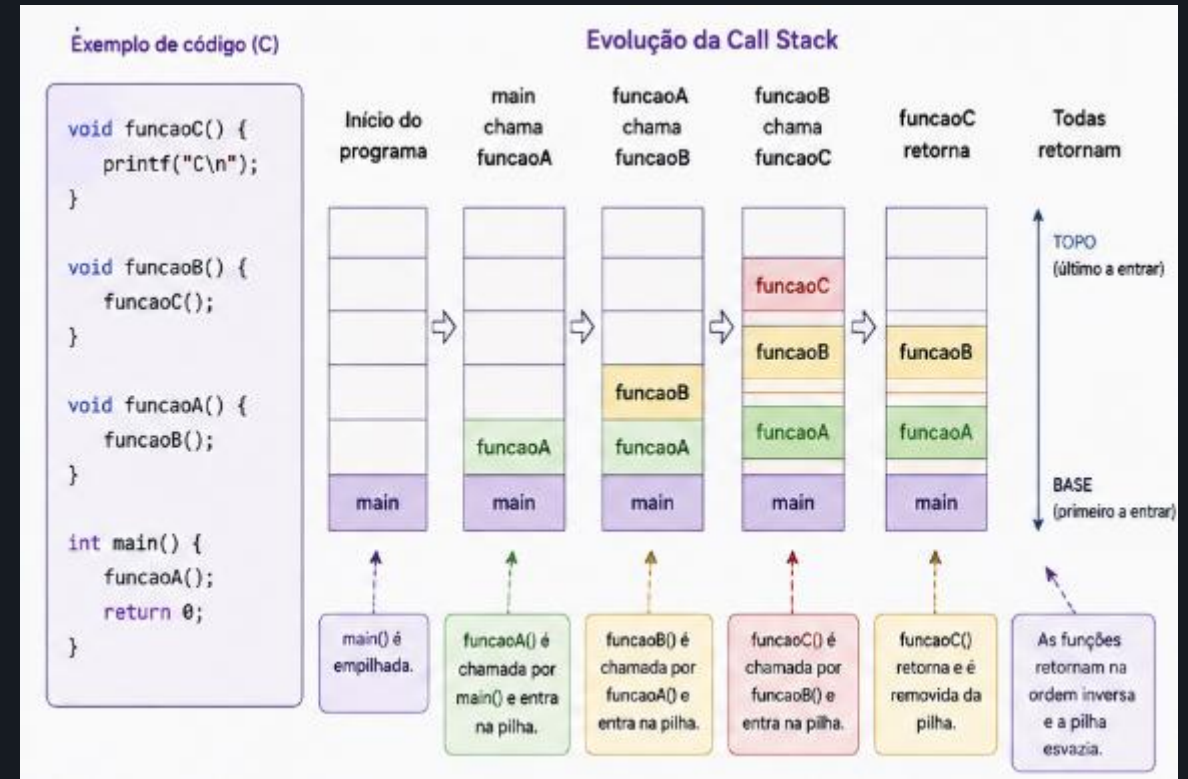
O Código

```
void contagem_regressiva(int n) {  
    // Caso Base (Parada)  
    if (n == 0) {  
        printf("Fogo!\n");  
        return;  
    }  
  
    printf("%d ... ", n);  
  
    // Chamada Recursiva  
    contagem_regressiva(n - 1);  
}
```

O Compilador e a Call Stack

O compilador C usa a Pilha de Chamadas (Call Stack). A cada nova chamada recursiva, ele empilha um bloco de memória com os parâmetros.

Quando o caso base é alcançado, o C desempilha as funções resolvendo tudo de trás para frente. Sem o caso base, ocorre o temido Stack Overflow!



A Anatomia de uma Função Recursiva



O Caso Base (Condição de Parada): É a regra vital. É a solução trivial para o problema. Sem ela, a função executa para sempre até o sistema operacional interromper com erro de memória.



A Chamada Recursiva: A função aciona a si mesma passando parâmetros que sempre modificam o estado atual em direção ao Caso Base.



O Desempilhamento (Backtracking): O momento em que o caso base retorna um valor, e o C propaga essa resposta de volta através de todas as instâncias da função empilhadas na memória.

Curiosidades do Mundo Dev

O Efeito Droste nas artes é a analogia visual perfeita da recursão. Na programação, a linguagem LISP (1958) focou pesadamente em chamadas recursivas no lugar de laços tradicionais.

Na cultura popular, a ideia de infinito recursivo sem um caso base gera a famosa anedota computacional de que há apenas 'Tartarugas até lá embaixo!'



A Terra gira
em torno do Sol.

Não, meu filho.
A Terra está apoiada
nas costas de uma
tartaruga gigante.

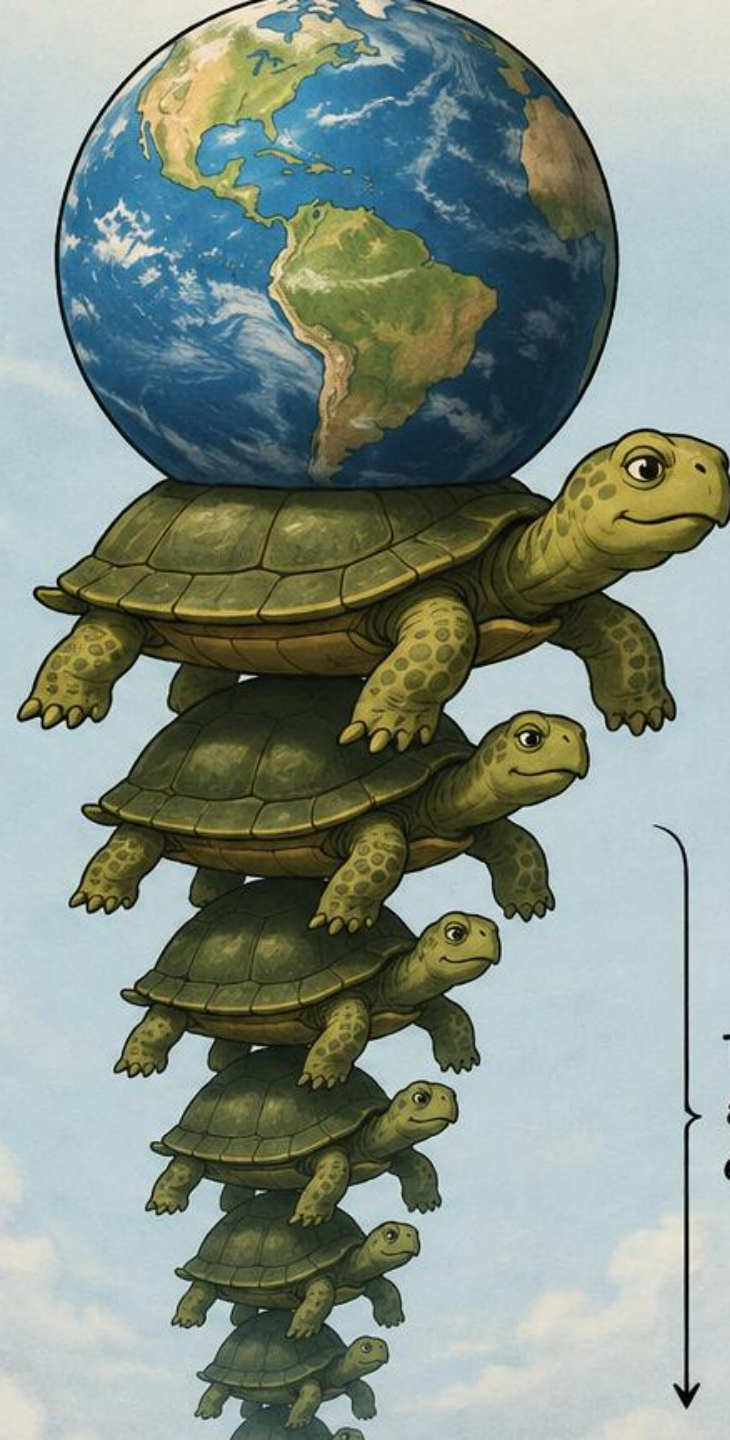
E em que essa
tartaruga está
apoiada?

Em outra
tartaruga.

E essa
outra
tartaruga?

Em outra
tartaruga.

Meu filho,
são tartarugas
até lá embaixo!



Tartarugas
até lá
embaixo!

Os 10 Problemas Clássicos

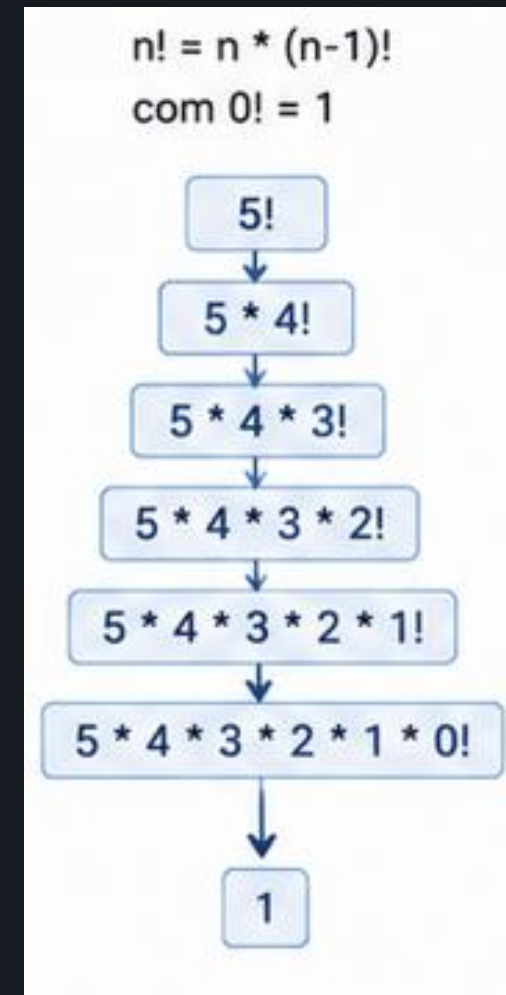
1. O Fatorial ($n!$)

A Base Matemática

O cálculo de um fatorial é considerado o "Hello World" da recursividade, pois sua própria definição matemática já utiliza a auto-referência estrutural.

$$n! = n \times (n - 1) !$$

A operação pausa e aguarda o cálculo do próximo número menor, criando uma corrente lógica. O encerramento definitivo (caso base) acontece quando se atinge $0!$, que, por definição matemática, é igual a 1.



Implementação: Fatorial

Análise do Algoritmo

A primeira verificação na função fatorial blinda a execução contra laços infinitos, retornando 1 imediatamente se n for 0.

Se não for o caso base, a linha final executa a recursão: a função multiplica o número n atual pelo retorno da próxima chamada da mesma função.

Código C

```
int fatorial(int n) {  
    // Caso Base  
    if (n == 0) {  
        return 1;  
    }  
  
    // Chamada Recursiva  
    return n * fatorial(n - 1);  
}
```

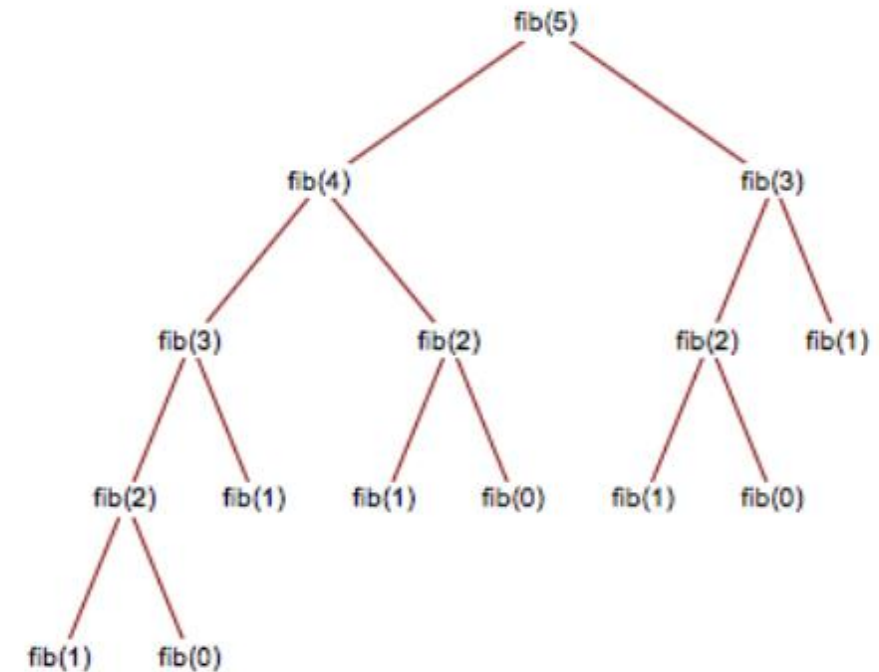
2. Sequência de Fibonacci

Múltiplas Chamadas Recursivas

Na famosa sequência de Fibonacci, o valor atual depende estritamente da soma dos dois números diretamente anteriores a ele.

$$F(n) = F(n - 1) + F(n - 2)$$

Diferente do fatorial, uma única invocação da função cria duas novas chamadas. Isso gera uma enorme árvore de execuções simultâneas empilhadas na memória.



Implementação: Fibonacci

Crescimento Exponencial

O caso base ocorre duplamente: tanto 0 quanto 1 retornam eles mesmos, resolvendo as pontas soltas da árvore lógica.

Atenção: embora elegante, essa abordagem bruta em C recalcula os mesmos nós várias vezes, tornando-se muito lenta para valores de n acima de 40.

Código C

```
int fibonacci(int n) {  
    // Caso Base Duplo  
    if (n ≤ 1) {  
        return n;  
    }  
  
    // Ramificação Recursiva Dupla  
    return fibonacci(n - 1) + fibonacci(n -  
2);  
}
```

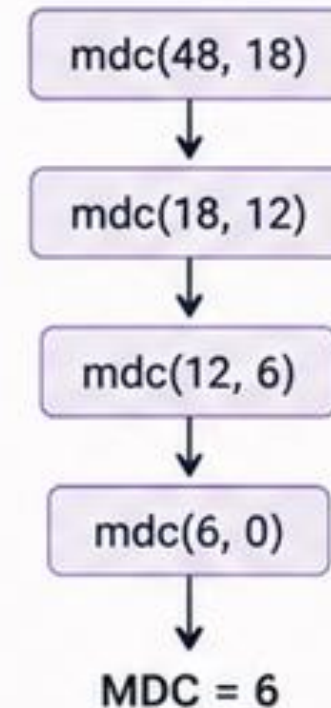

3. Máximo Divisor Comum (MDC)

O Algoritmo de Euclides

Este antigo método grego encontra o maior número que divide dois inteiros sem deixar restos. É incrivelmente rápido graças à sua natureza auto-redutora.

A matemática estabelece que o MDC entre A e B é perfeitamente igual ao MDC entre B e o **resto da divisão de A por B**. O processo afunila o problema a cada passo.

Algoritmo de Euclides:
 $\text{mdc}(a, b) = \text{mdc}(b, a \bmod b)$
até $b = 0$



Implementação: MDC

Recursividade Pura

Não há a necessidade de variáveis complexas. O caso base identifica o fim do caminho: quando o segundo número fornecido for exatamente 0.

O operador módulo (%) em C é o verdadeiro herói matemático nesta operação, permitindo o colapso rápido dos valores iterativos.

Código C

```
int mdc(int a, int b) {  
    // Caso Base: resto zerado  
    if (b == 0) {  
        return a;  
    }  
  
    // Passa 'b' e o resto de 'a' por 'b'  
    return mdc(b, a % b);  
}
```

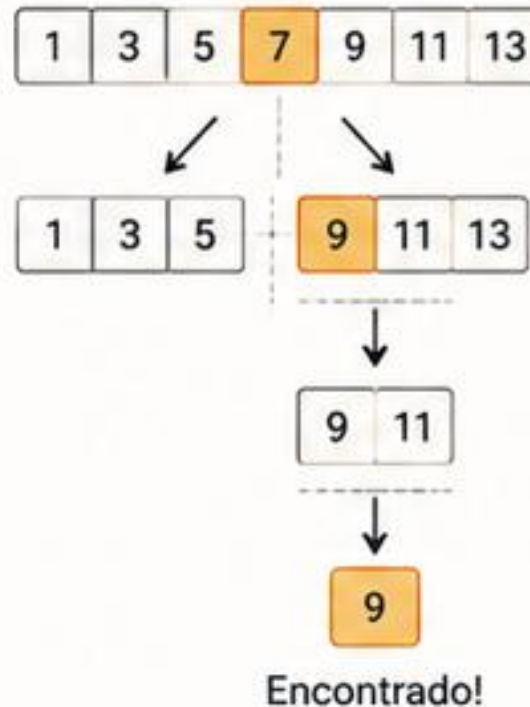
4. Busca Binária (Binary Search)

Dividir para Conquistar

Tentar achar uma palavra específica em um dicionário página por página seria terrível. O instinto humano é abrir o livro ao meio, avaliar se a palavra deve estar antes ou depois, e repetir a quebra no meio.

A busca binária recursiva converte exatamente este raciocínio lógico em código de máquina veloz operando em arrays pré-ordenados.

Divide o vetor ao meio e busca na metade correta.



Implementação: Busca Binária

Gerenciamento de Índices

A função recebe 4 valores: o array, o índice da esquerda, o índice da direita, e o alvo. O ponto meio dita o caso base de sucesso.

A recursividade atua ignorando inteiramente metade do array, passando limites atualizados (meio + 1 ou meio - 1) para a próxima iteração na *Call Stack*.

Código C

```
int buscaBinaria(int arr[], int esq, int
dir, int x) {
    if (dir ≥ esq) {
        int meio = esq + (dir - esq) / 2;
        if (arr[meio] == x) return meio; //
        Encontrou!

        if (arr[meio] > x)
            return buscaBinaria(arr, esq, meio-
1, x);

        return buscaBinaria(arr, meio+1, dir,
x);
    }
    return -1; // Não encontrado
}
```

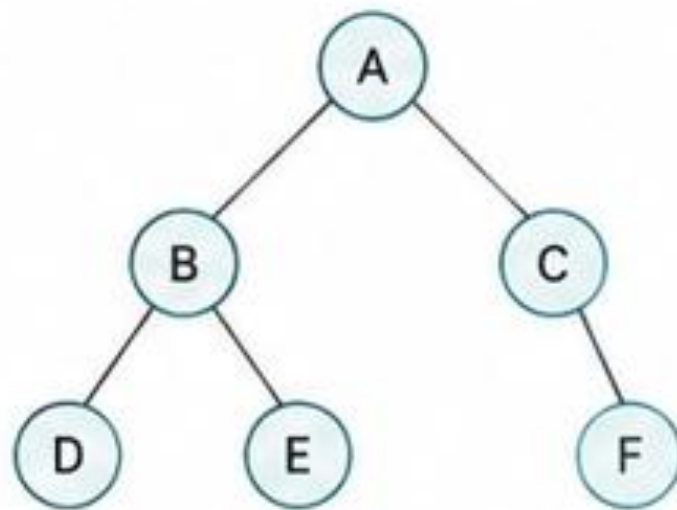
5. Travessia de Árvores Binárias

Uma Estrutura Naturalmente Recursiva

Diferente de arrays de linha única, árvores de dados abrigam estruturas bidimensionais. Cada "nó" central tem potencialmente o seu próprio ramo esquerdo e direito, que por sua vez contêm mais nós.

Iterar isso com um clássico laço for exige manipulação agressiva de ponteiros e pilhas. A recursão lida com as ramificações de forma orgânica e em três linhas lógicas.

A visita a um nó e suas subárvores é feita recursivamente.



Exemplo: Pré-ordem
(A, B, D, E, C, F)

Implementação: In-Order Traversal

Impressão Ordem Simétrica

O caso base é alcançado sempre que o ponteiro mira para NULL (o fim de um ramo).

Ao chamarmos a própria função para o lado esquerdo, imprimirmos o nó atual com printf, e na sequência chamarmos o lado direito, conseguimos magicamente extrair todos os dados em ordem alfabética ou numérica estrita.

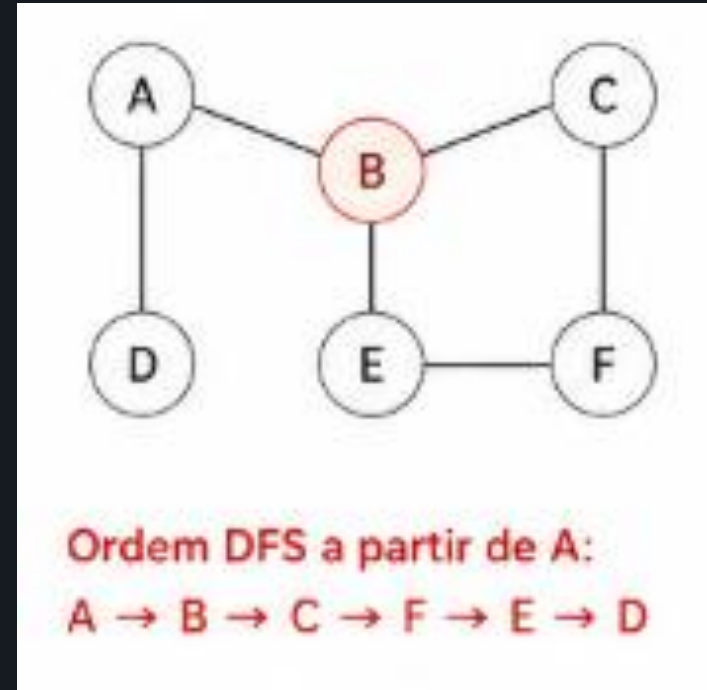
Código C

```
struct Node {  
    int data;  
    struct Node* left;  
    struct Node* right;  
};  
  
void inOrder(struct Node* root) {  
    if (root != NULL) {  
        inOrder(root→left);      // Vai a  
esquerda  
        printf("%d ", root→data); //  
Imprime centro  
        inOrder(root→right);     // Vai a  
direita  
    }  
}
```


6. Busca em Profundidade (DFS)

A Busca em Profundidade (DFS) em grafos mergulha o mais fundo possível em um caminho antes de voltar (backtrack).

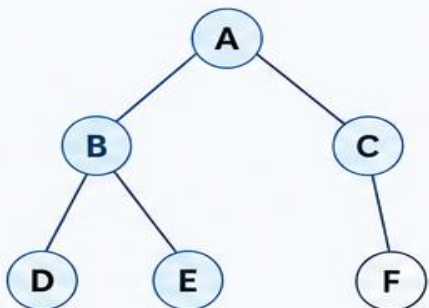
Para cada vizinho não visitado, a função chama a si mesma. É uma abordagem elegante que substitui completamente o uso de pilhas manuais por pilhas gerenciadas pelo compilador C.



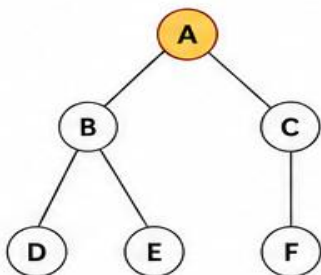
BUSCA EM PROFUNDIDADE (DFS)

Explora o mais profundo possível em cada ramo antes de voltar atrás (backtrack).

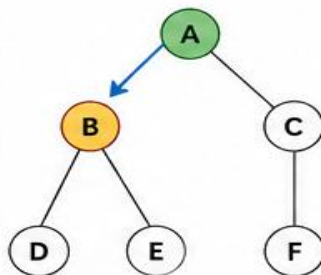
GRAFO



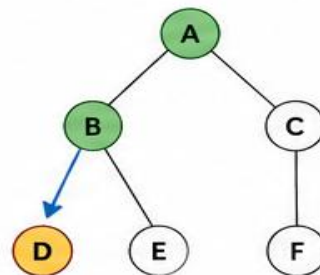
1 Comece no nó A.



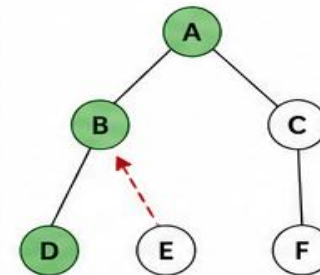
2 Vá para o primeiro vizinho de A: B.



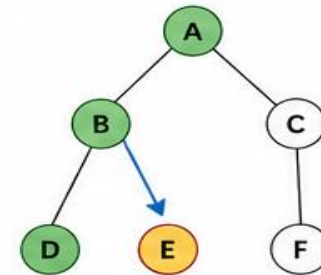
3 Vá para o primeiro vizinho de B: D.



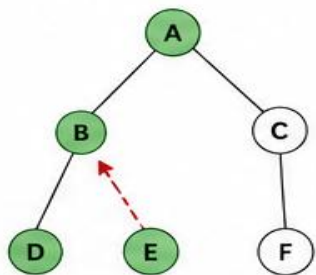
4 D não tem vizinhos não visitados. Volte para B.



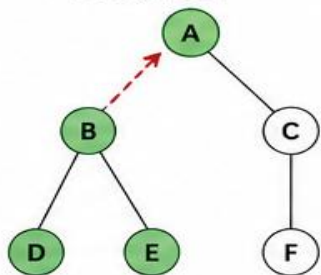
5 Próximo vizinho de B: E.



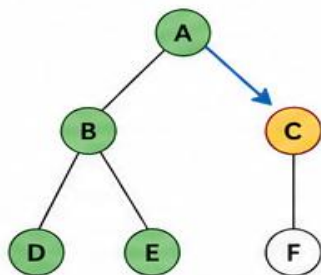
6 E não tem vizinhos não visitados. Volte para B.



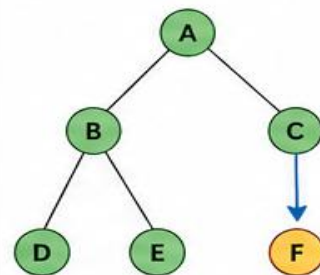
7 B não tem mais vizinhos não visitados. Volte para A.



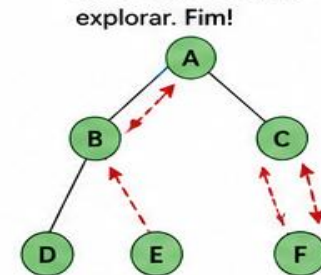
8 Próximo vizinho de A: C.



9 Vá para o primeiro vizinho de C: F.



10 F não tem vizinhos não visitados. Volte até não haver mais nós para explorar. Fim!



- = Visitado
- = Nó atual
- = Backtrack (voltando)
- = Explorando

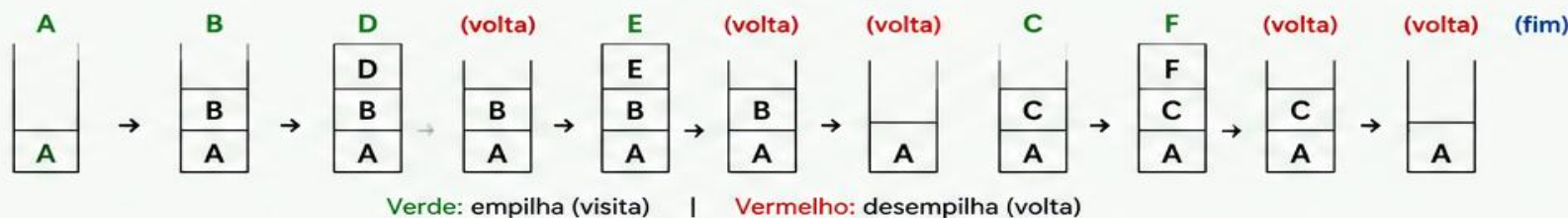
Ordem de visita (exemplo iniciando em A):

A → B → D → E → C → F

COMO FUNCIONA

- Escolha um nó inicial (A).
- Visite um vizinho não visitado e continue a partir dele.
- Quando não houver mais vizinhos não visitados, volte (backtrack) para o nó anterior.
- Continue até visitar todos os nós alcançáveis.

Pilha de Chamadas (Call Stack) - DFS



Implementação: Grafo DFS

Protegendo Contra Ciclos

Em grafos, caminhos fechados e cíclicos são normais. Por isso o código marca imediatamente o nó em um array `visitado[]`. Sem isso, a recursão entraria em looping e travaria o sistema.

Dentro do laço `for`, qualquer vizinho detectado que ainda não esteja marcado como visitado ativa imediatamente um novo mergulho profundo.

Código C

```
void DFS(int vertice) {
    visitado[vertice] = 1; // Marca nó
    seguro
    printf("%d ", vertice);

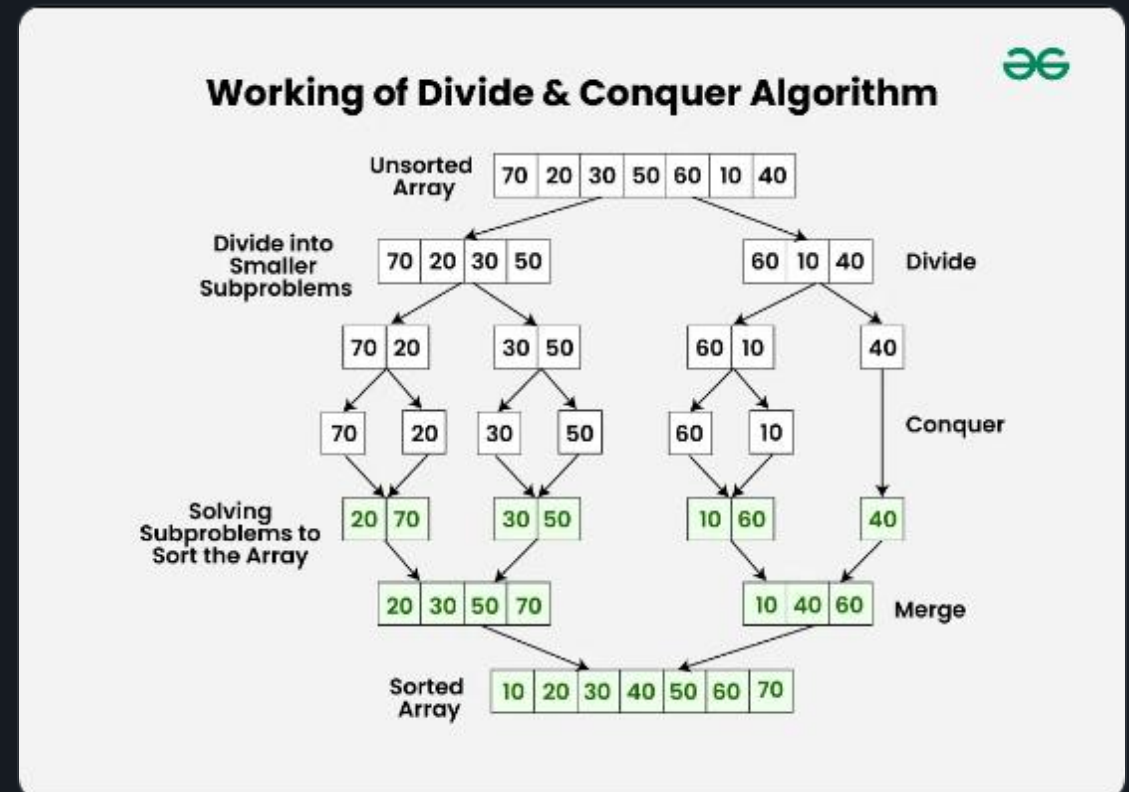
    // Investiga caminhos possíveis
    for(int i = 0; i < vizinhos; i++) {
        int viz = grafo[vertice][i];
        if(!visitado[viz]) {
            DFS(viz); // Aprofunda a busca
            recursiva!
        }
    }
}
```

7. Merge Sort (Ordenação por Mistura)

O Domínio da Desmontagem

Considerado um dos mais potentes algoritmos de ordenação, o Merge Sort prova que organizar muitos dados é difícil, mas organizar dois pequenos grupos é trivial.

A recursividade ataca cortando as peças do array repetidamente ao meio. O caso base atinge arrays contendo um elemento apenas, o qual já está tecnicamente ordenado. O trabalho pesado acontece na "volta", costurando as metades de volta na ordem correta.

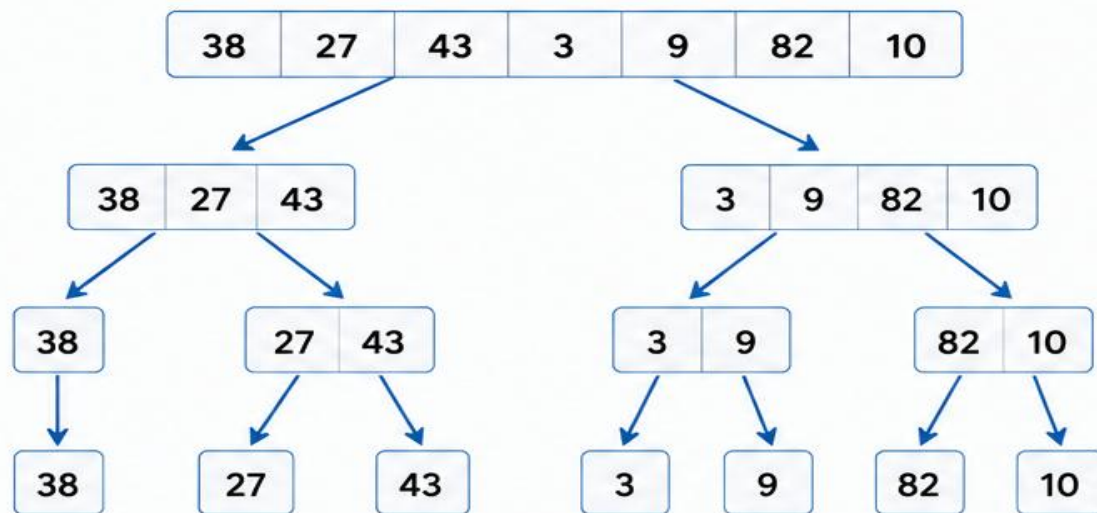


MERGE SORT (Dividir para Conquistar)

Ordena uma lista dividindo-a em partes menores, ordenando cada parte e depois intercalando-as.

1. DIVIDIR

Divida a lista ao meio recursivamente até que cada sublista tenha apenas 1 elemento.

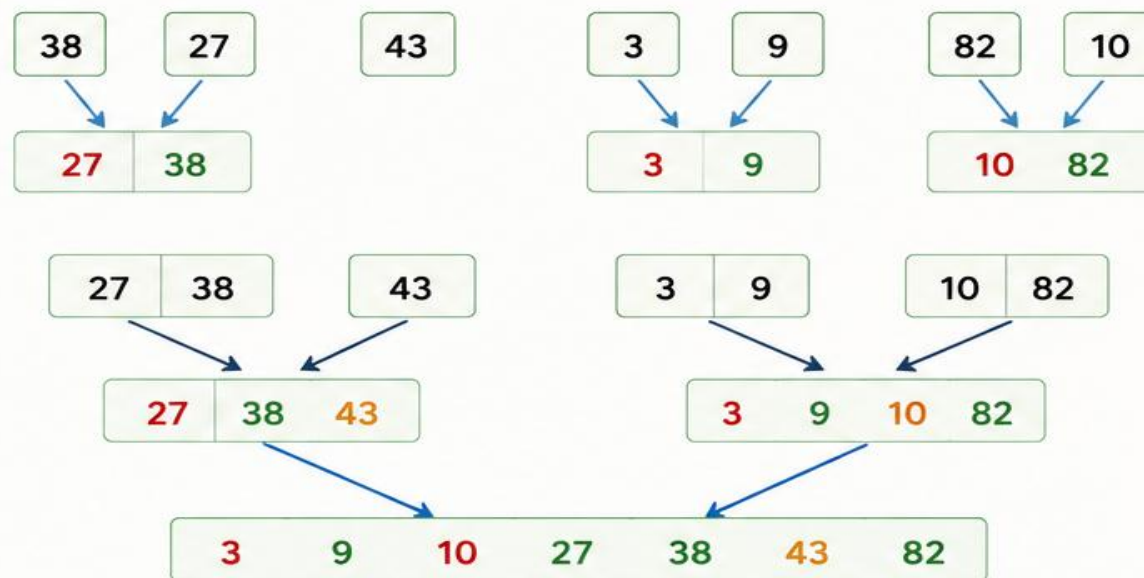


Caso base:

Sublistas com 1 elemento já estão ordenadas.

2. CONQUISTAR (INTERCALAR / MERGE)

Intercale as sublistas ordenadas, sempre escolhendo o menor elemento de cada sublista.



Regra do merge: Compare os primeiros elementos das sublistas, coloque o menor no resultado e avance na sublista correspondente.



COMO FUNCIONA

1. Divida a lista ao meio até chegar em listas de 1 elemento.
2. Intercale as listas ordenadas de baixo para cima.
3. O resultado final será a lista totalmente ordenada.



COMPLEXIDADE

- Melhor caso: $O(n \log n)$
- Caso médio: $O(n \log n)$
- Pior caso: $O(n \log n)$
- Espaço adicional: $O(n)$



Vantagens:

É eficiente, estável e funciona bem para grandes volumes de dados.

Implementação: Merge Sort

Invocação Dupla

O algoritmo executa de forma muito similar à busca binária quanto à matemática dos recortes dos dados.

Porém, as duas chamadas a mergeSort dividem a totalidade da array de entrada. Somente depois que os retalhos alcançam o tamanho 1 a robusta função externa de ordenação, conhecida como merge(), entra em atuação efetiva.

Código C

```
void mergeSort(int arr[], int esq, int dir)
{
    if (esq < dir) {
        int m = esq + (dir - esq) / 2;

        // Divide em sub-arrays
        mergeSort(arr, esq, m);
        mergeSort(arr, m + 1, dir);

        // Combina os lados agora ordenados
        merge(arr, esq, m, dir);
    }
}
```

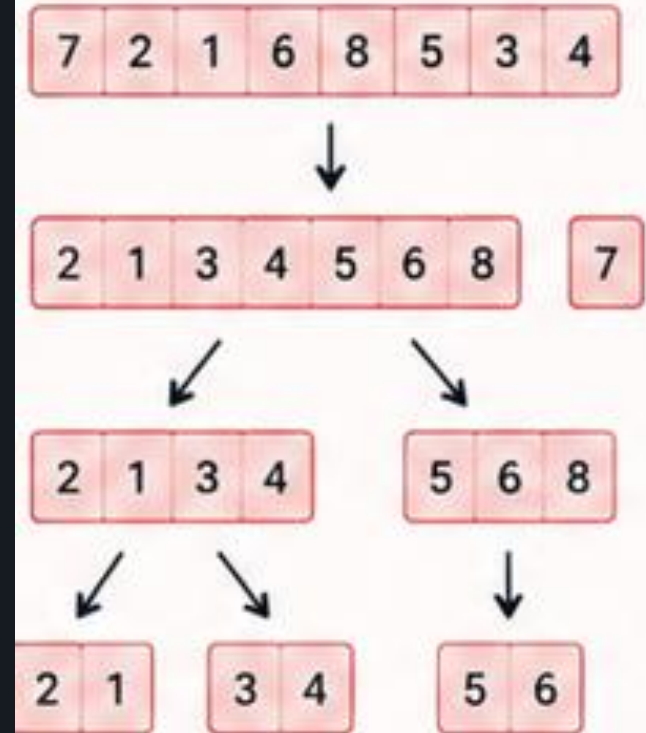
8. Quick Sort (Ordenação Rápida)

O Poder do Pivô Central

Diferente do algoritmo anterior onde as partes são trabalhadas após o fatiamento máximo, o Quick Sort processa os números de modo imediato.

Ele identifica um número de referência batizado de "Pivô". Ao atirar valores maiores para um lado e menores ao oposto, assegura que este ponto específico encontra seu posicionamento permanente, acionando recursões nas metades geradas em seguida.

Escolhe um pivô, particiona menores e maiores e ordena recursivamente.





Divide e Conquista
Rápido na prática
e in-place (usa pouca
memória extra).

QUICK SORT

Ideia principal: escolha um pivô e reorganize os elementos de modo que os menores fiquem à esquerda e os maiores à direita. Depois, aplique o mesmo processo recursivamente nas partes.

1 Escolha um pivô

Escolhemos o último elemento como pivô.



2 Particionamento

Reorganize o array: menores que o pivô à esquerda e maiores à direita.



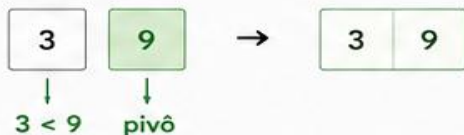
3 Recursão

Aplique o mesmo processo nas partes esquerda e direita.



4 Ordenando o subarray esquerdo [3, 9]

Pivô = 9



Subarrays: [] e [3] → já estão ordenados.

5 Ordenando o subarray direito [27, 43, 82, 38]

Pivô = 38



Pivô = 43



Subarrays: [] e [27] → ordenados.

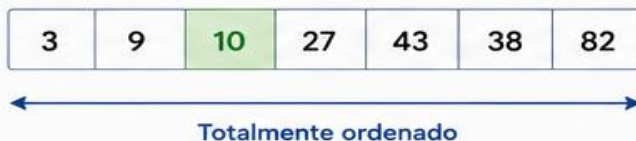
Pivô = 82



Subarrays: [] e [] → ordenados.

6 Combine tudo

Os subarrays já estão ordenados, então o array final é:



COMO FUNCIONA

- Escolha um pivô (ex.: último elemento).
- Particione o array:
 - Coloque os menores que o pivô à esquerda.
 - Coloque os maiores que o pivô à direita.
 - Deixe o pivô na posição correta.
- Aplique recursivamente o Quick Sort nas partes esquerda e direita.
- Pare quando o subarray tiver 0 ou 1 elemento.

PSEUDOCÓDIGO

```
quickSort(A, baixo, alto)
  if baixo < alto:
    p = particionar(A, baixo, alto)
    quickSort(A, baixo, p-1)
    quickSort(A, p+1, alto)
```

```
particionar(A, baixo, alto)
  pivô = A[alto]
  i = baixo - 1
  for j = baixo até alto-1:
    if A[j] <= pivô:
      i = i + 1
      trocar A[i] e A[j]
  trocar A[i+1] e A[alto]
  return i + 1 // posição do pivô
```



COMPLEXIDADE

- Melhor caso: $O(n \log n)$
- Caso médio: $O(n \log n)$
- Pior caso: $O(n^2)$ (ex.: array já ordenado)
- Espaço adicional: $O(\log n)$ (recursão)



OBSERVAÇÕES

- Muito rápido na prática.
- Desempenho pior quando o pivô é sempre o menor ou o maior elemento.
- Escolher um pivô aleatório reduz esse risco.

Implementação: Quick Sort

Elegância com Partições

A arquitetura em C chama primeiro a complexa lógica do método acessório `particiona()`, que devolve onde exatamente o vetor dividiu o pivô em duas seções definitivas.

Somente então a recursão invoca as mesmas instruções exatas para as peças isoladas na esquerda (menores) e na direita (maiores), ordenando o conjunto in-loco (na memória).

Código C

```
void quickSort(int arr[], int baixo, int
alto) {
    if (baixo < alto) {
        // Ponto pivô, esquerda menor,
        direita maior
        int pivo_idx = particiona(arr,
baixo, alto);

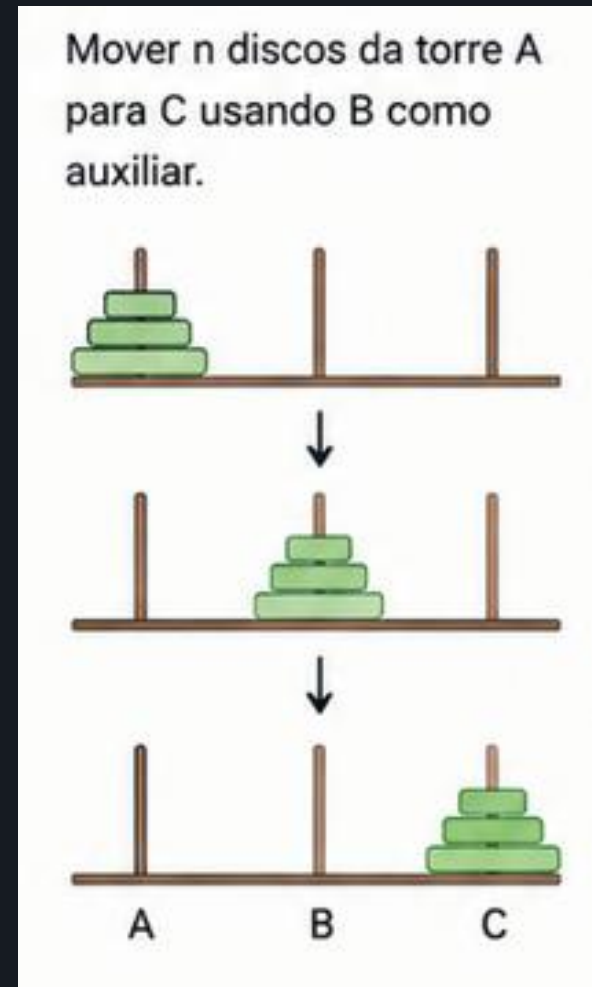
        // Recursão autônoma das metades
        resultantes
        quickSort(arr, baixo, pivo_idx - 1);
        quickSort(arr, pivo_idx + 1, alto);
    }
}
```

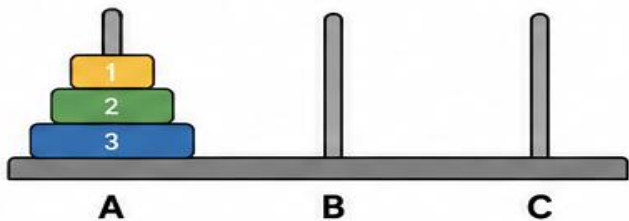
9. Torres de Hanói

O Quebra-Cabeça Infinito

Como mover uma enorme torre em forma piramidal entre pinos, sem depositar componentes maiores sobre peças menores em nenhum momento?

A visão recursiva soluciona transferindo provisoriamente quase todos os discos ($N-1$) para a área de descanso. Depois, movimenta de forma definitiva a matriz principal da base para seu alvo. O passo subsequente desloca o pilar reserva montando-o na peça fundamental.





TORRE DE HANÓI

Objetivo: mover todos os discos da Torre A para a Torre C usando a Torre B como auxiliar, seguindo as regras.



REGRAS

1. Mover apenas um disco por vez.
2. Um disco maior nunca pode ficar sobre um menor.
3. Só é permitido mover o disco do topo de uma torre.

IDEIA DO ALGORITMO (RECUSIVO)

Para mover n discos de A para C usando B:

1. Mover os $n-1$ discos de A para B (usando C).
2. Mover o maior disco (n) de A para C.
3. Mover os $n-1$ discos de B para C (usando A).

CASO BASE

Se houver apenas 1 disco:
mova diretamente da origem para o destino.

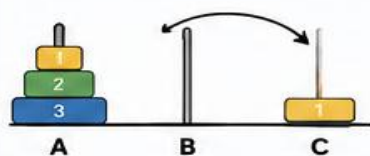


COMPLEXIDADE

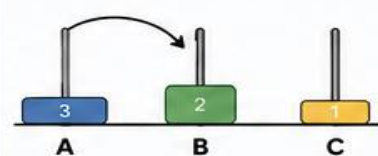
- Número mínimo de movimentos: $2^n - 1$
- Exemplo: com 3 discos $\rightarrow 2^3 - 1 = 7$ movimentos

EXEMPLO PASSO A PASSO (3 DISCOS)

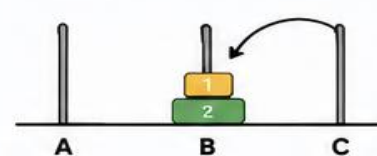
- 1 Mover disco 1 de A $\rightarrow$ C



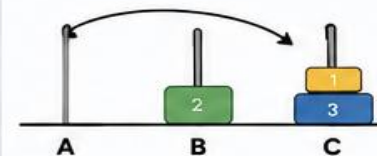
- 2 Mover disco 2 de A $\rightarrow$ B



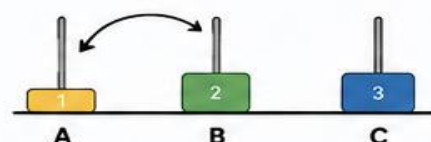
- 3 Mover disco 1 de C $\rightarrow$ B



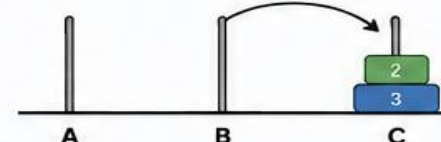
- 4 Mover disco 3 de A $\rightarrow$ C



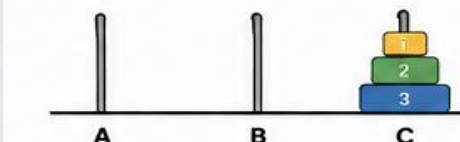
- 5 Mover disco 1 de B $\rightarrow$ A



- 6 Mover disco 2 de B $\rightarrow$ C

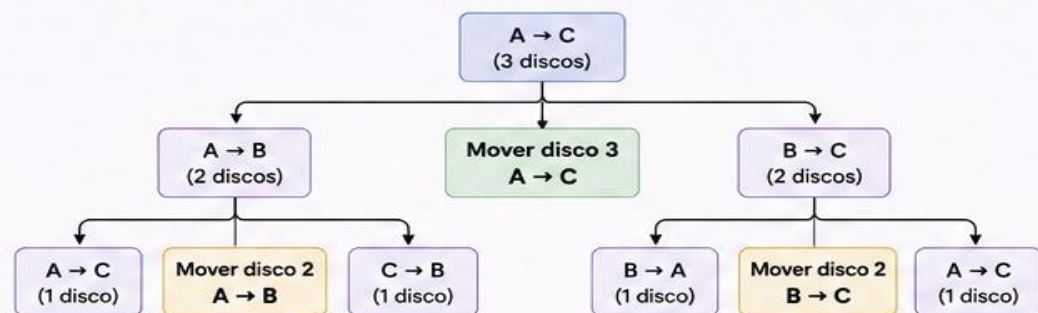


- 7 Mover disco 1 de A $\rightarrow$ C



Resultado: todos os discos foram movidos de A para C em 7 movimentos ($2^3 - 1$).

COMO O ALGORITMO FUNCIONA (EXEMPLO COM 3 DISCOS)



Legenda:

A = Origem
B = Auxiliar
C = Destino



Cada chamada recursiva resolve um subproblema menor.

PSEUDOCÓDIGO

```
hanoi(n, origem, auxiliar, destino)
  se n == 1 então
    mover disco de origem para destino
  senão
    hanoi(n-1, origem, destino, auxiliar)
    // 1. mover n-1 para auxiliar
    mover disco de origem para destino
    // 2. mover o maior disco
    hanoi(n-1, auxiliar, origem, destino)
    // 3. mover n-1 para destino
  fim-se
```

Parâmetros:

- n = número de discos
- origem = torre inicial
- auxiliar = torre auxiliar
- destino = torre final

Chamadas iniciais:

```
hanoi(3, A, B, C)
// mover 3 discos de A para C
// usando B como auxiliar
```

Implementação: Torres de Hanói

A Alternância dos Pinos

Note como a função recebe os caracteres identificadores das colunas, variando quem será considerado destino e auxiliador em cada fase do trajeto complexo.

Uma instrução printf estática é injetada no interstício da chamada, ditando qual componente desliza na iteração observada e documentando passos no monitor.

Código C

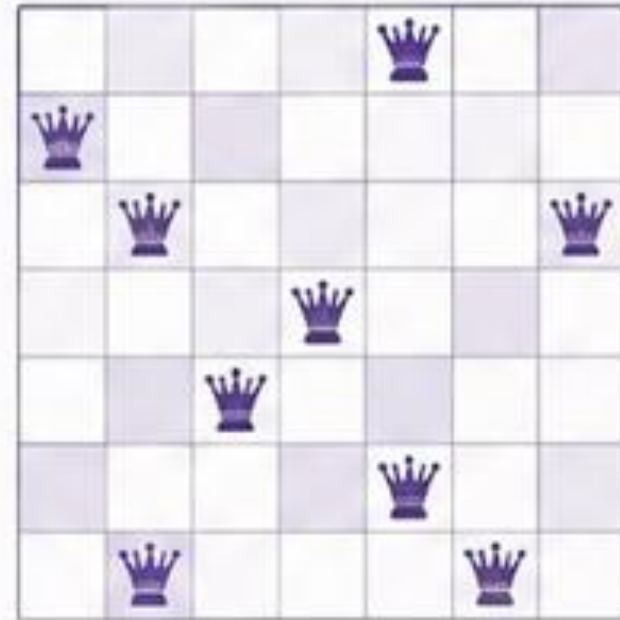
```
void hanoi(int n, char org, char dst, char aux) {  
    if (n == 1) { // Apenas um disco livre  
        printf("Mover disco de %c para %c\n", org, dst);  
        return;  
    }  
  
    // Desloca bloco principal fora do caminho  
    hanoi(n-1, org, aux, dst);  
    printf("Mover placa %d de %c para %c\n", n, org, dst);  
    // Acomoda os discos reservados na base segura  
    hanoi(n-1, aux, dst, org);  
}
```

10. Problema das 8 Rainhas (Backtracking)

Colocar rainhas sem ataques mútuos requer Backtracking. A recursividade tenta posicionar uma rainha e chama a próxima linha.

Se der errado, a função desfaz a jogada e tenta a próxima posição possível. É uma árvore exaustiva de tentativas e erros estruturada organicamente na memória interna.

Colocar 8 rainhas no tabuleiro 8x8 sem que se ataquem.



PROBLEMA DAS 8 RAINHAS

Colocar 8 rainhas em um tabuleiro 8x8 de forma que nenhuma rainha ataque outra.
Rainhas se atacam na mesma linha, coluna ou diagonal.



IDEIA

Colocamos uma rainha por coluna, da esquerda para a direita.
Para cada coluna, testamos todas as linhas possíveis.
Se uma posição não for segura, fazemos backtracking e tentamos outra linha.

COMO VERIFICAR SE É SEGURO?

- ✓ Nenhuma outra rainha na mesma linha à esquerda.
- ✓ Nenhuma outra rainha na diagonal superior esquerda.
- ✓ Nenhuma outra rainha na diagonal inferior esquerda.

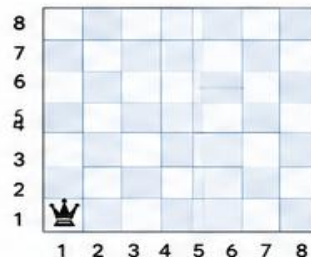


COMPLEXIDADE

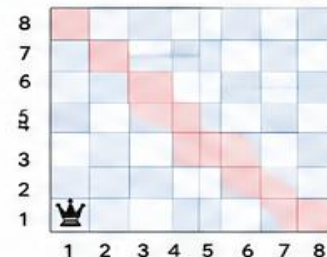
- Espaço: $O(n)$ (profundidade da recursão)
 - Tempo: $O(n!)$ (pior caso)
- n = número de rainhas (aqui, $n = 8$)

EXEMPLO DE EXECUÇÃO (BACKTRACKING)

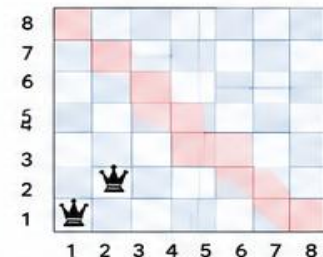
1 Coluna 1
Tenta linha 1



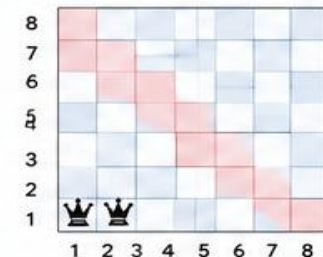
2 Coluna 2
Tenta linha 1 (não segura)



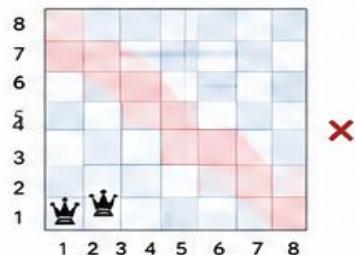
3 Coluna 2
Tenta linha 3 (segura)



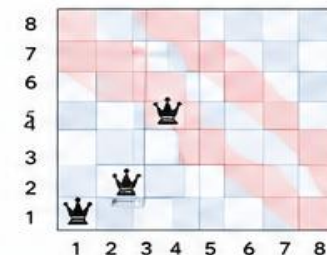
4 Coluna 3
Tenta linha 1 (não segura)



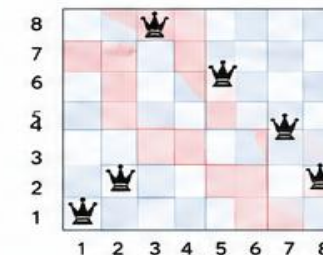
5 ... continua tentando outras linhas na coluna 3



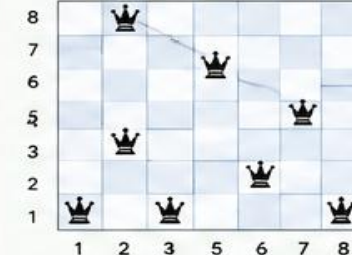
6 Coluna 3
Tenta linha 5 (segura)



7 ... continua até coluna 8



8 Solução encontrada!



LEGENDA



= Rainha

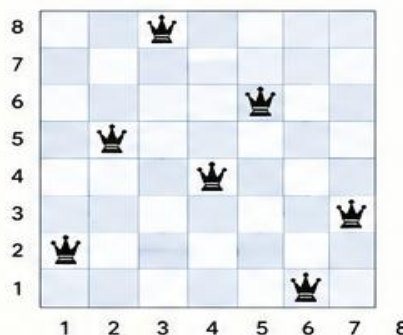


= Célula disponível



= Célula atacada

SOLUÇÃO FINAL (uma das possíveis)



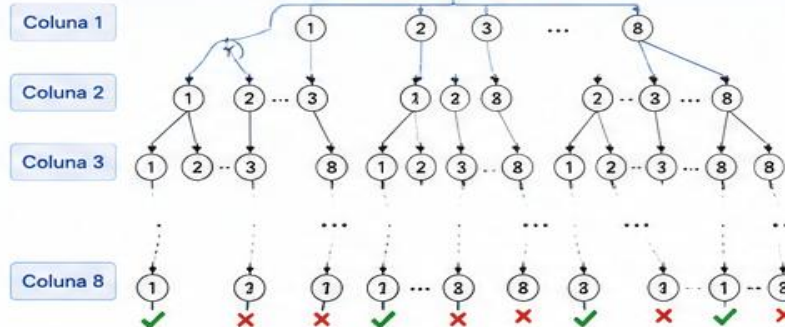
POSIÇÕES DAS RAINHAS (linha, coluna)

(1,1), (5,2), (8,3), (6,4),
(2,5), (7,6), (3,7), (6,8)



Existem 92 soluções diferentes para o problema das 8 rainhas.

ÁRVORE DE BUSCA (IDEIA)



RESUMO

1. Coloque uma rainha na coluna 1.
2. Para cada coluna seguinte, tente todas as linhas.
3. Se for seguro, vá para a próxima coluna.
4. Se não for, volte (backtrack) e tente outra linha.
5. Quando chegar à coluna 8, encontrou uma solução.

Implementação: N-Rainhas

Tentativa, Erro e Reversão

O poder irrestrito do "Backtracking" vive após o escopo do chamado condicional if. Se ele falha determinando um impasse no jogo inteiro...

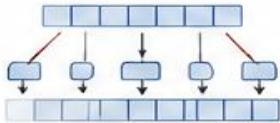
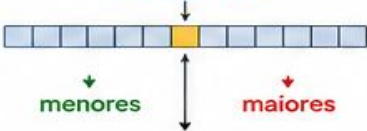
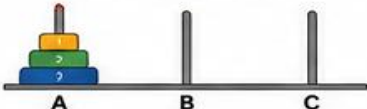
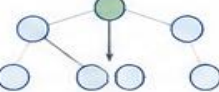
A matriz global assume um reajuste com board = 0. O sistema retrocede a configuração ao instante passado desatando as amarras mal formadas. A lógica varre silenciosa opções remanescentes através do C.

Código C

```
int resolverRainhas(int board[N][N], int col) {  
    if (col ≥ N) return 1; // Colocamos todas as rainhas!  
  
    for (int i = 0; i < N; i++) {  
        if (seguro(board, i, col)) {  
            board[i][col] = 1; // Ocupa a casa  
  
            if (resolverRainhas(board, col + 1))  
                return 1;  
  
            board[i][col] = 0; // BACKTRACK: desfaz jogada  
        }  
    }  
    return 0;  
}
```


COMPLEXIDADE ASSINTÓTICA DE ALGORITMOS

Análise de tempo de execução (notação Big O)

ALGORITMO	IDEIA PRINCIPAL	COMPLEXIDADE ASSINTÓTICA			ESPAÇO EXTRA	ESTÁVEL?	OBSERVAÇÕES
		MELHOR CASO	CASO MÉDIO	PIOR CASO			
MERGE SORT 	Divide o array em metades, ordena recursivamente e intercala as partes ordenadas.	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓ Sim	- Desempenho consistente em todos os casos. - Requer memória extra para a intercalação.
QUICK SORT 	Escolhe um pivô, particiona em menores e maiores e aplica recursivamente nas partes.	$O(n \log n)$ (pivô balanceado)	$O(n \log n)$	$O(n^2)$ (pivô ruim)	$O(\log n)$ médio $O(n)$ pior caso	✗ Não	- Muito rápido na prática. - Pior caso ocorre quando o pivô é sempre o menor ou o maior elemento.
TORRE DE HANÓI 	Move n discos da torre A para a torre C usando a torre B como auxiliar, seguindo as regras.	$O(2^n)$	$O(2^n)$	$O(2^n)$	$O(n)$ (na recursão)	—	Exponencial: cresce muito rápido com o número de discos.
BUSCA EM PROFUNDIDADE (DFS) 	Explora o mais profundo possível em cada ramo antes de voltar atrás (backtracking).	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$ (pilha de chamadas)	—	- V = vértices, E = arestas. - Usa pilha (explícita ou recursão).
PROBLEMA DAS 8 RAINHAS (BACKTRACKING) 	Coloca 1 rainha por coluna, testando posições seguras e fazendo backtracking quando necessário.	$O(1)$ (encontrou logo a solução)	$O(N!)$ aproximado	$O(N!)$	$O(N)$ (na recursão)	—	- N = tamanho do tabuleiro (para 8 rainhas, N=8). - Crescimento fatorial.

★

LEGENDA

n

= tamanho do input (vetor, número de discos, etc.)

V

= número de vértices do grafo

E

= número de arestas do grafo

N

= tamanho do tabuleiro (N = 8 para 8 rainhas)

CRESCIMENTO DAS FUNÇÕES (do menor para o maior)

O(1)

constante

<

O(log n)

logarítmico

<

O(n)

linear

<

O(n log n)

linear logarítmico

<

O(n²)

quadrático

<

O(2^n)

exponencial

<

O(n!)

fatorial

OBSERVAÇÕES GERAIS

Melhor caso: cenário mais favorável.

Caso médio: comportamento esperado na maioria das entradas.

Pior caso: limite superior de tempo.

Espaço extra: memória adicional usada além do input.